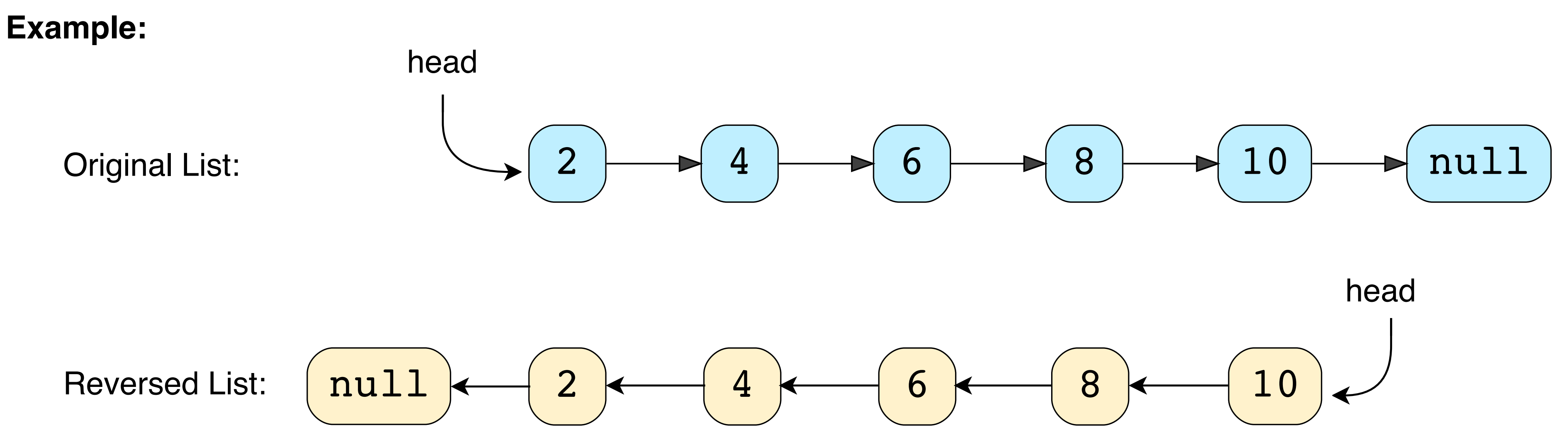
Solution: Reverse a LinkedList

Problem Statement

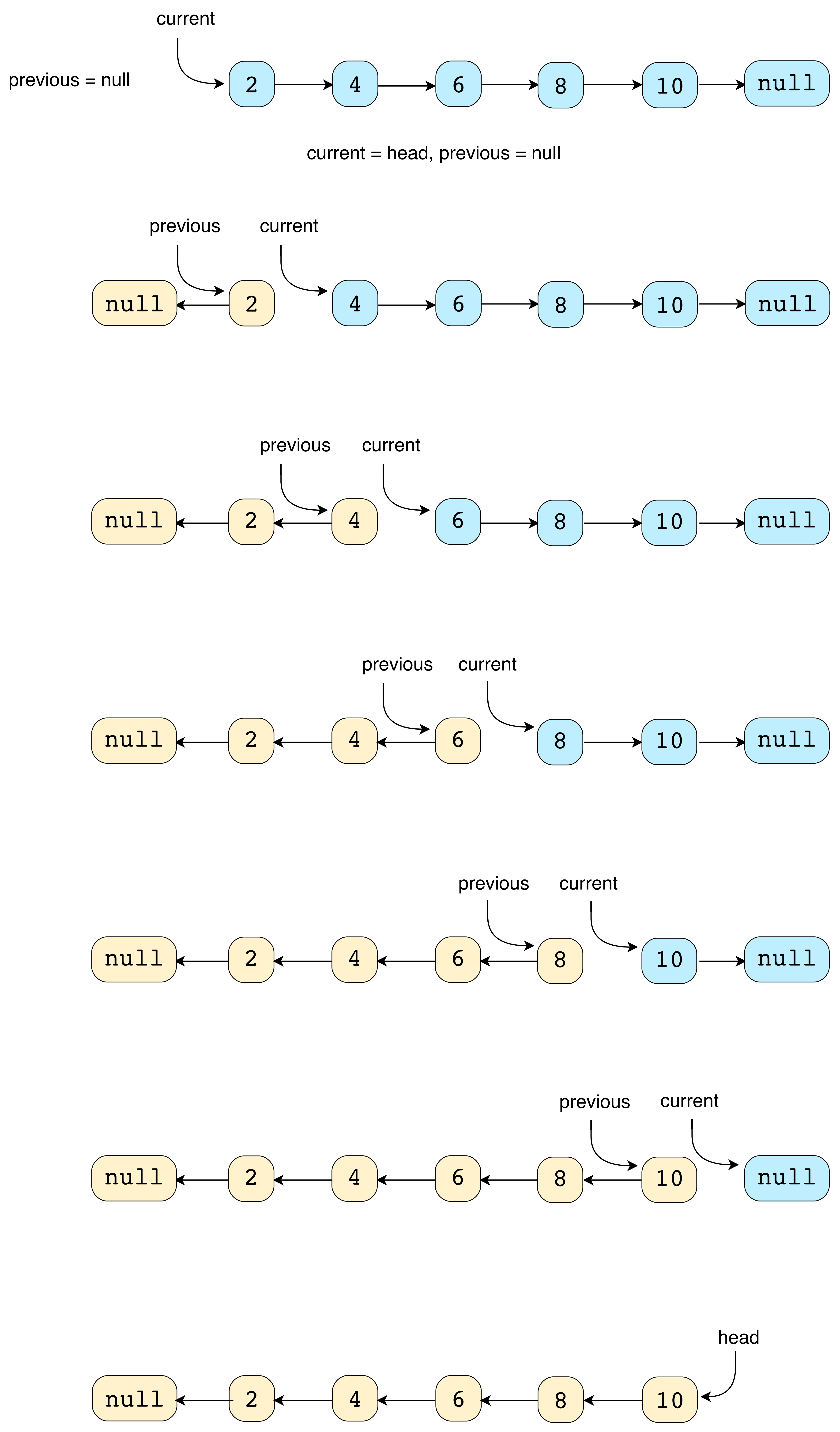
Given the head of a Singly LinkedList, reverse the LinkedList. Write a function to return the new head of the reversed LinkedList.



Solution

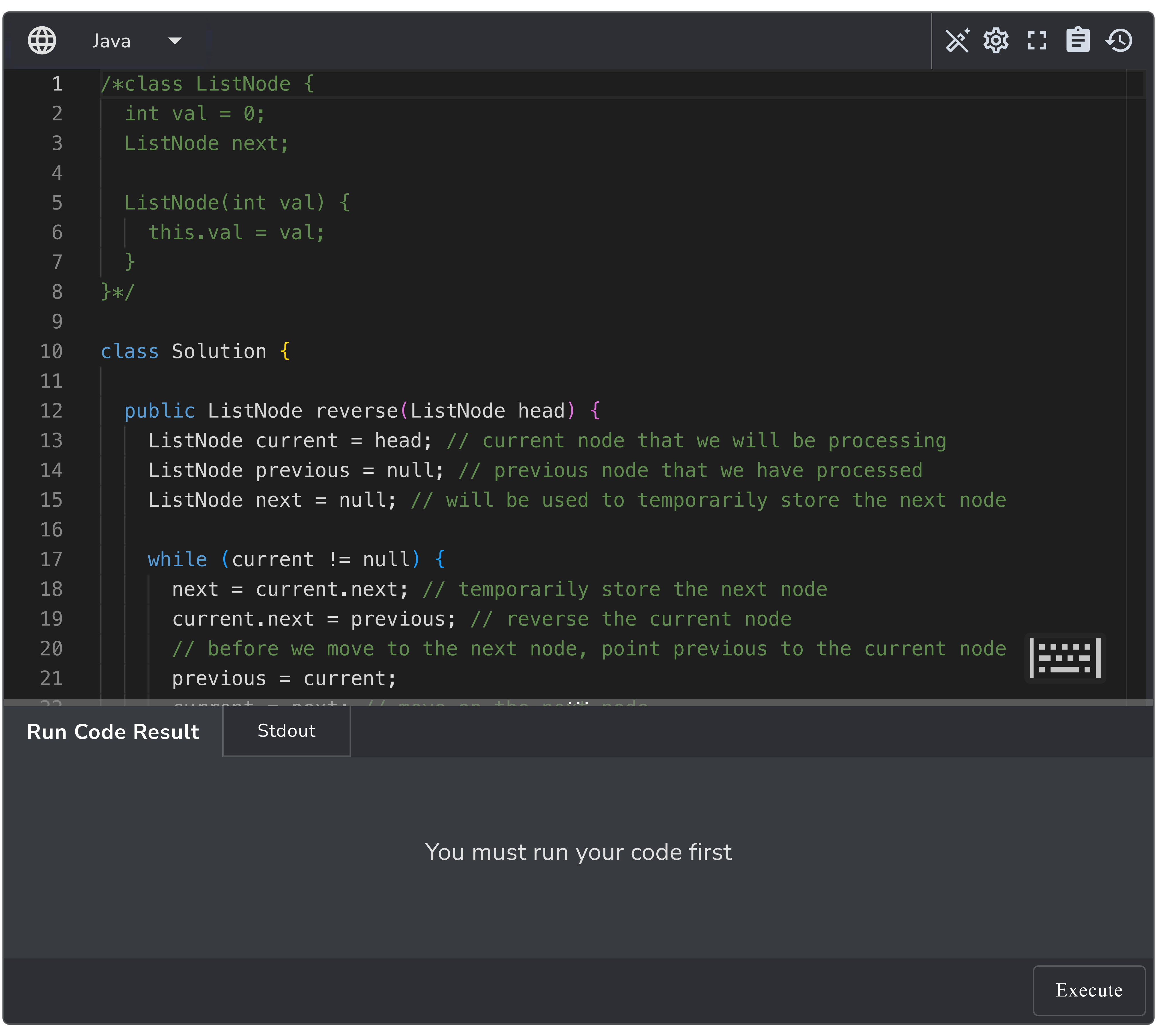
To reverse a LinkedList, we need to reverse one node at a time. We will start with a variable `current` which will initially point to the head of the LinkedList and a variable `previous` which will point to the previous node that we have processed; initially `previous` will point to null.

In a stepwise manner, we will reverse the `current` node by pointing it to the `previous` before moving on to the next node. Also, we will update the previous to always point to the `previous` node that we have processed. Here is the visual representation of our algorithm:



Code

Here is what our algorithm will look like:



```java
/*class ListNode {
  int val = 0;
  ListNode next;

  ListNode(int val) {
    this.val = val;
  }
}*/

class Solution {

  public ListNode reverse(ListNode head) {
    ListNode current = head; // current node that we will be processing
    ListNode previous = null; // previous node that we have processed
    ListNode next = null; // will be used to temporarily store the next node

    while (current != null) {
      next = current.next; // temporarily store the next node
      current.next = previous; // reverse the current node
      // before we move to the next node, point previous to the current node
      previous = current;
```

You must run your code first

Time Complexity

The time complexity of our algorithm will be O(N) where 'N' is the total number of nodes in the LinkedList.

Space Complexity

We only used constant space, therefore, the space complexity of our algorithm is O(1).

← Previous
Reverse A LinkedList (Easy)

Next →
Maximum Number Of Balloons (Easy)

☐ Mark as Completed